

Data Collection Request — message to the data / collection team

Subject: Storage metadata collection (multi-OS) — what to collect and in which format

Hi,

We are scoping an AI / agentic system for **storage capacity management and deviation detection** across our heterogeneous fleet of **500+ servers** (AIX, all Linux distributions, and Windows Server — all treated as first-class and delivered together). The system collects **non-sensitive storage metadata**, compares each host against a per-OS reference standard, detects deviations, forecasts saturation, and proposes prioritized remediations.

To get there, I need richer, structured data about the servers. This message describes **what to collect** and **in which format**. The five objectives the data must serve are:

- **01 — Deviation detection vs standards:** spot servers whose partitioning / filesystem layout differs from the model expected for their OS and version.
- **02 — Optimization (useless / obsolete / misplaced):** reclaim space — obsolete data, over-provisioned volumes, application data sitting on system partitions.
- **03 — Reliability & full-filesystem prevention:** detect ahead of time the filesystems about to fill up (in **bytes AND inodes**) to avoid incidents.
- **04 — Operational efficiency & forecasting:** standardize storage structures and anticipate growth / capacity needs.
- **05 — Risk reduction & compliance:** ensure only appropriate data resides on system partitions; check mount options / ACLs (CIS baselines).

What I need as output (best effort)

1. **ONE envelope file per collection run** — a single object `{ snapshot_id, generated_at, host_count, hosts: [ ... ] }`, where each element of `hosts[]` is **one server's document** following the **single unified schema** in Appendix A (the same schema for every platform). For very large fleets, the same content as **JSONL** (one host object per line) is accepted — in that case each line carries its own `snapshot_id + scan_timestamp` so the lines remain attributable to a run.
2. **All sizes in EXACT BYTES** (integers) — never rounded, no KB/MB/GB, no blocks, no percentages. Native KB/block outputs (`df -k`, `lsps`, `vgs --units b`) are normalized to bytes at collection.
3. **Per filesystem: total, used, AND available (all three)** in bytes — plus **inodes** (`inodes_total`, `inodes_used`) wherever they exist (Unix). Windows has no inodes, so those fields are `null` there.
4. **A UTC (ISO-8601) timestamp + a snapshot_id per run**, with collection **repeated over time** — append, never overwrite. The time series is what makes growth and saturation forecasting possible.
5. **Preserve the filesystem ↔ topology link** via the `backing_ref` / `group_ref` keys (linkage by keys, not by nesting). This is what lets the pipeline derive "system" vs "data" downstream.
6. **Non-sensitive metadata only** — no filenames, no owners, no file contents. On Windows, provide an **ACL summary** (inheritance flag, ACE count, well-known principals, a count of non-well-known ACEs, and an "everyone-write" flag) — never owners or named ACLs / SIDs.

7. **Any field not applicable to an OS/fstype = explicit null** (verified N/A), **never omitted** — so we can distinguish "N/A" from "not collected". The collection command branches per platform; the target schema does not.

Proposed approach

To validate the format quickly and cheaply, let's **start with a representative sample** spanning several server types — e.g. one or two of each: AIX (rootvg + datavg), RHEL/Oracle Linux (XFS, with and without LVM), Debian/Ubuntu (ext4), SUSE/SLES (btrfs with subvolumes), and Windows Server (basic disk + a Storage Spaces pool). Once that sample round-trips cleanly through the schema, we scale out to the full fleet on a repeated (e.g. daily) cadence.

A few collection notes that matter for byte accuracy:

- **AIX** — LVM (rootvg/datavg), JFS2; `lsvg / lslv / lspv`, `df -k / df -v`, `lspv -a`. No ext-style reserved blocks → `reserved_bytes = null`.
- **RHEL / Oracle Linux / Rocky / AlmaLinux** — XFS by default → no `reserved_bytes (null)`; use `df -i` for inodes; LVM common (`vgs / lvs / pvs --units b`). `/boot/efi` is present only on UEFI x86_64.
- **Debian / Ubuntu** — ext4 by default → `reserved_bytes` via `tune2fs -l` (reserved blocks × block size); LVM optional.
- **SUSE / SLES** — btrfs by default (subvolumes + snapshots) → `df` is misleading; use `btrfs filesystem usage` / `qgroups` for the real byte figures; LVM optional.
- **Windows Server** — NTFS/ReFS via WinRM + PowerShell; no inodes (`inodes_* = null`); ACLs replace mount options (`access_model = "ntfs-acl"`, `mount_options = null`, `acl_summary` populated). Storage Spaces: pool = `group (storage-pool)`, virtual disk = `volume (virtual-disk)`. Basic disk: partitions exposed as `volumes (partition)`, `groups` empty. Sizes are already in bytes.

A reminder on the collection / processing boundary: **storage_class (system vs data), percentages, growth rates, deviations, and severities are all DERIVED downstream** by the pipeline — please do **not** compute or collect them. Collect only the raw fields below.

Appendix A — expected JSON format

Obligation legend: **REQ** = required · **REC** = recommended · **COND** = conditional (nullable per OS/fstype) · **REQ*** = required once a group/volume exists (else the key is `null`).

`snapshot_id`, `generated_at`, and `host_count` live at the **envelope / run level**. Each host element carries its own `scan_timestamp`; it does **not** carry its own `snapshot_id` (the run identifier is the envelope's; for JSONL, see note 1 above).

```
// — THE FILE: one envelope object for the whole run —————
{
  "snapshot_id": "string",          // REQ — collection-run identifier (RUN level)
  "generated_at": "string",          // REQ — UTC ISO-8601, file generation time (RUN level)
  "host_count": number,              // REQ — number of elements in hosts[] (RUN level)
  "hosts": [ /* ONE document per server — schema below */ ]
}

// — ONE ELEMENT OF hosts[]: a single server (OS-agnostic, all OS) —————
{
  // Identity & collection metadata
  "host_id": "string", // REQ — stable host identifier
  "hostname": "string", // REQ
  "gsma_code": "string", // REQ — site/fleet code
  "host_role_env": "string", // REQ — prod | preprod | test | dev ...
  "platform": "string", // REQ — enum: aix | linux | windows
  "os_family": "string", // REQ — AIX | RedHat | Debian | Suse | OracleLinux | Windows
  "os_distribution": "string", // REQ — AIX | RHEL | Ubuntu | Debian | SLES | OracleLinux |
WindowsServer
  "os_version": "string", // REQ
  "architecture": "string", // REQ — ppc64 | x86_64 | ...
  "scan_timestamp": "string", // REQ — UTC ISO-8601 (this host's own scan time)
  "collection_method": "string", // REQ — ssh-shell | winrm-powershell | agent ...
  "scan_confidence": number, // REQ — 0..1 collection reliability

  // UNIVERSAL DENOMINATOR: every mountable Unix filesystem / Windows volume
  "filesystems": [{
    "mount_point": "string", // REQ — "/", "/var" | "C:", "D:" | mount-folder
    "label": "string|null", // REC
    "fstype": "string", // REQ — xfs|ext4|btrfs|jfs2|ntfs|refs|vfat...
    "size_total_bytes": number, // REQ — bytes
    "size_used_bytes": number, // REQ — bytes
    "size_available_bytes": number, // REQ — bytes
    "reserved_bytes": "number|null", // COND — ext only; null on xfs/jfs2/ntfs/refs/btrfs
    "inodes_total": "number|null", // COND — null on Windows NTFS/ReFS
    "inodes_used": "number|null", // COND — null on Windows NTFS/ReFS
    "access_model": "string", // REQ — "posix-mount-options" | "ntfs-acl"
    "mount_options": "string|null", // COND — Unix flags; null on Windows
    "acl_summary": "object|null", // COND — non-sensitive Windows ACL summary; null on
Unix
    "backing_kind": "string", // REQ — lvm-lv | partition | virtual-disk | network |
pseudo
    "backing_ref": "string|null", // REQ* — key to the underlying volume/disk
    "group_ref": "string|null", // REQ* — key to the group (VG/pool)
    "content_aggregates": { /* object */ } // REC — non-sensitive aggregates (2nd pass)
  }],

```

```

// PLATFORM STORAGE TOPOLOGY – neutral levels + *_kind discriminator
// Empty for plain-partition Linux or basic-disk Windows (no pool).
"storage_topology": {
  "groups": [{
    "group_name":      "string",          // REQ* (once a group exists)
    "group_kind":      "string",          // REQ* – lvm-vg | storage-pool
    "size_total_bytes": number,           // REQ* – bytes
    "size_free_bytes": number,           // REQ* – bytes
    "member_disk_count": number,         // REQ*
    "state":           "string",          // REQ* – active | online | degraded ...
    "extent_size_bytes": "number|null",   // COND – PE size (LVM) / extent; null if N/A
    "capabilities": {                    // COND – nullable per OS
      "max_lvs":       "number|null",     // AIX
      "max_pps":       "number|null",     // AIX
      "vg_type":       "string|null",     // AIX (big | scalable ...)
      "resiliency":    "string|null"      // Windows (Mirror | Parity | Simple)
    }
  }],
  "volumes": [{
    "volume_name": "string",             // REQ* (once a volume exists)
    "volume_kind": "string",             // REQ* – lvm-lv | partition | virtual-disk
    "group_ref":   "string|null",        // REQ* – key to parent group
    "size_bytes":  number,               // REQ* – bytes
    "type":        "string|null",        // REC – jfs2 | linear | striped | mirror | gpt-part
    ...
    "redundancy":  "string|null"        // COND – none | mirror | parity ...
  }],
  "disks": [{
    "disk_name":  "string",              // REQ* (if the disk is exposed)
    "size_bytes": number,                // REQ* – bytes
    "free_bytes": "number|null",         // REC – bytes
    "group_ref":  "string|null",         // REC – key to group
    "media_type": "string|null"          // REC – hdd | ssd | unspecified
  }]
},

// PAGING / SWAP / PAGEFILE
"paging": [{
  "name":      "string",                // REQ
  "kind":      "string",                // REQ – aix-paging-lv | linux-swap-partition | linux-
swap-lv | linux-swap-file | windows-pagefile
  "size_total_bytes": number,           // REQ – bytes
  "size_used_bytes":  "number|null"     // REC – bytes
}]
}

```

REQ* means required but **nullable** when the topology does not exist (plain-partition Linux, basic-disk Windows): the key is then `null` rather than omitted.

Appendix B — fields to collect: what, why, objective served

Objectives recap: **01** deviation detection vs standards · **02** optimization (useless/obsolete/misplaced) · **03** reliability & full-filesystem prevention · **04** operational efficiency & forecasting · **05** risk reduction & compliance.

File / run level

Field	What it is / why it matters	Objectives
<code>snapshot_id</code> / <code>generated_at</code> / <code>host_count</code>	Identify the collection run and anchor the time series (run id, UTC generation time, host count).	03, 04

Host level (one per server)

Field	What it is / why it matters	Objectives
<code>host_id</code>	Stable key to de-duplicate and correlate the same host over time.	03, 04
<code>hostname</code>	Identification in reports and dashboards.	04
<code>gsma_code</code>	Account/fleet binding → reference-model choice & normalization.	01, 05
<code>host_role_env</code>	prod/test → weights incident severity.	03, 05
<code>platform</code> / <code>os_family</code> / <code>os_distribution</code> / <code>os_version</code>	Select the right standard (RHEL8 ≠ RHEL9 ≠ AIX ≠ Windows ...).	01
<code>architecture</code>	Drives conditional rules (UEFI /boot/efi , ppc64 ...).	01
<code>scan_timestamp</code>	UTC time base for this host → growth forecasting.	03, 04
<code>collection_method</code>	Data traceability / quality of the collection.	01, 04
<code>scan_confidence</code>	Collection completeness → reliability of downstream results.	01

`filesystems[]` level (each Unix filesystem / Windows volume — the universal denominator)

Field	What it is / why it matters	Objectives
<code>mount_point</code>	Path semantics (<code>/var</code> , <code>C:</code> ...) → classification & comparison vs standard.	01, 02, 05
<code>label</code>	Correlation / readability.	01
<code>fstype</code>	FS type → applicable rules, exclusion of pseudo/network filesystems.	01
<code>size_total_bytes</code>	Capacity → basis for utilization and forecasting.	03
<code>size_used_bytes</code>	Usage → imminent saturation signal + optimization basis.	02, 03
<code>size_available_bytes</code>	Real remaining headroom.	03
<code>reserved_bytes</code>	System reserve (ext) that skews perceived usage.	03
<code>inodes_total</code> / <code>inodes_used</code>	Inode exhaustion even without byte saturation (Unix).	03
<code>access_model</code>	Distinguishes Unix (mount options) vs Windows (ACL) compliance.	05
<code>mount_options</code>	Compliance flags (<code>ro/nodev/nosuid/noexec</code> ...).	01, 05
<code>acl_summary</code>	Windows compliance posture (non-sensitive, non-nominative summary).	05
<code>backing_kind</code> / <code>backing_ref</code>	Link the FS to its underlying volume/LV/partition.	01, 02
<code>group_ref</code>	Link the FS to its group (VG/pool) → derive system vs data downstream.	01, 02, 05
<code>content_aggregates</code>	"Misplaced / obsolete" signals (non-sensitive aggregates, 2nd pass).	02, 05

`storage_topology.groups[]` level (VG / Storage Pool)

Field	What it is / why it matters	Objectives
<code>group_name</code>	Identifies the group (<code>rootvg/datavg</code> , pool).	01, 02, 05
<code>group_kind</code>	Type discriminator (<code>lvm-vg</code> / <code>storage-pool</code>).	01
<code>size_total_bytes</code> / <code>size_free_bytes</code>	Group capacity & growth headroom.	03
<code>member_disk_count</code>	Resilience (a single-disk group is a single point of failure).	01, 03
<code>state</code>	Group health (<code>inactive</code> / <code>degraded</code>).	03
<code>extent_size_bytes</code>	Allocation granularity (consistency check).	01
<code>capabilities</code>	Ceilings & resilience (AIX <code>max_lvs/max_pps/vg_type</code> ; Windows resiliency).	01, 03

`storage_topology.volumes[]` level (LV / virtual disk / partition)

Field	What it is / why it matters	Objectives
<code>volume_name</code>	Extension target; FS ↔ group link.	O1, O3
<code>volume_kind</code>	Type discriminator (lvm-lv / partition / virtual-disk).	O1
<code>group_ref</code>	Binding to the parent group.	O1
<code>size_bytes</code>	Allocated size (possible gap vs the filesystem).	O1, O3
<code>type</code>	Segment/volume type (jfs2 / linear / striped / gpt-part ...).	O1, O5
<code>redundancy</code>	Mirror / parity → reliability.	O1, O3

`storage_topology.disks[]` level (PV / physical disk)

Field	What it is / why it matters	Objectives
<code>disk_name</code>	Disk ↔ group correlation.	O1, O3
<code>size_bytes</code> / <code>free_bytes</code>	Physical capacity & extension headroom.	O3
<code>group_ref</code>	Binding to the group.	O1
<code>media_type</code>	hdd / ssd (performance context).	O4

`paging[]` level (swap / paging space / pagefile)

Field	What it is / why it matters	Objectives
<code>name</code> / <code>kind</code>	Paging space / swap / pagefile identity and type.	O1, O3
<code>size_total_bytes</code> / <code>size_used_bytes</code>	Size & memory pressure.	O3

The reference standards / golden images, the OS-by-OS field-applicability matrix, the field-by-field rationale plus the Ansible collection playbook, the LangGraph pipeline architecture, and a machine-validatable JSON Schema file are **upcoming** and will be shared separately. For now, the sample collection above is the immediate ask.

Thanks — happy to walk through the schema or the sample on a call.